

Gold artifact – Coding

Case study: Appointment Booking System

1. Cel

Referencyjna implementacja logiki rezerwacji wizyt, zgodna z gold requirements i gold architecture, ale zawierająca kontrolowaną złożoność pozwalającą na ocenę jakości testów, wykrywania błędów i analizy ograniczeń LLM.

2. Zakres implementacji

- Implementacja obejmuje:
- przeglądanie dostępnych slotów,
- rezerwację slotu,
- anulowanie rezerwacji,
- limit aktywnych rezerwacji na użytkownika,
- wykrywanie konfliktów czasowych między wizytami tego samego użytkownika,
- zarządzanie statusem slotów,
- obsługę blokady slotu,
- rejestrowanie zdarzeń audytowych,
- podstawową kontrolę właściciela rezerwacji.
- Nie obejmuje:
- płatności,
- powiadomień,
- panelu UI,
- stref czasowych,
- pełnej produkcyjnej obsługi transakcji rozproszonych.

3. Minimalne endpointy / operacje

- `get_available_slots(specialist_id, date)`
- `book_slot(user_id, slot_id)`
- `cancel_booking(user_id, booking_id)`
- `get_user_bookings(user_id)`
- `get_audit_log()`

4. Reguły biznesowe

BR1

Slot może zostać zarezerwowany tylko, jeśli ma status AVAILABLE.

BR2

Jeden użytkownik może mieć maksymalnie 3 aktywne rezerwacje.

BR3

Ten sam slot nie może zostać zarezerwowany dwa razy.

BR4

Użytkownik nie może mieć dwóch aktywnych rezerwacji o nakładających się przedziałach czasowych.

BR5

Anulowanie jest możliwe tylko dla własnej rezerwacji.

BR6

Anulowanie jest możliwe najpóźniej 24 godziny przed terminem.

BR7

Po anulowaniu slot nie zawsze wraca do AVAILABLE; jeśli slot został zablokowany administracyjnie, jego status pozostaje BLOCKED.

BR8

Każda operacja rezerwacji i anulowania musi zostać zapisana w logu audytowym.

BR9

Implementacja może zawierać uproszczenia techniczne, ale nie może naruszać podstawowych reguł biznesowych.

5. Referencyjna struktura kodu

booking_system/

├— app.py

├— models.py

├— repository.py

├— booking_service.py

├— schedule_service.py

├— audit_service.py

└— tests/

6. Referencyjna logika implementacyjna

models.py

```
from dataclasses import dataclass
```

```
from datetime import datetime  
from enum import Enum
```

```
class SlotStatus(str, Enum):  
    AVAILABLE = "AVAILABLE"  
    BOOKED = "BOOKED"  
    BLOCKED = "BLOCKED"  
    COMPLETED = "COMPLETED"
```

```
class BookingStatus(str, Enum):  
    BOOKED = "BOOKED"  
    CANCELLED = "CANCELLED"
```

```
@dataclass
```

```
class Slot:  
    id: int  
    specialist_id: int  
    start_time: datetime  
    end_time: datetime  
    status: SlotStatus = SlotStatus.AVAILABLE
```

```
@dataclass
```

```
class Booking:  
    id: int
```

```
user_id: int
slot_id: int
status: BookingStatus
created_at: datetime
```

```
@dataclass
```

```
class AuditEvent:
```

```
    id: int
    event_type: str
    user_id: int
    slot_id: int
    timestamp: datetime
    details: str = ""
```

```
repository.py
```

```
from datetime import datetime
```

```
from models import Slot, Booking, AuditEvent, BookingStatus
```

```
class InMemoryRepository:
```

```
    def __init__(self):
        self.slots = {}
        self.bookings = {}
        self.audit_events = {}
        self._booking_id = 1
        self._audit_id = 1
```

```
    def add_slot(self, slot: Slot):
```

```
self.slots[slot.id] = slot
```

```
def get_slot(self, slot_id: int):
```

```
    return self.slots.get(slot_id)
```

```
def save_slot(self, slot: Slot):
```

```
    self.slots[slot.id] = slot
```

```
def create_booking(self, user_id: int, slot_id: int):
```

```
    booking = Booking(
```

```
        id=self._booking_id,
```

```
        user_id=user_id,
```

```
        slot_id=slot_id,
```

```
        status=BookingStatus.BOOKED,
```

```
        created_at=datetime.utcnow(),
```

```
    )
```

```
    self.bookings[self._booking_id] = booking
```

```
    self._booking_id += 1
```

```
    return booking
```

```
def get_booking(self, booking_id: int):
```

```
    return self.bookings.get(booking_id)
```

```
def save_booking(self, booking: Booking):
```

```
    self.bookings[booking.id] = booking
```

```
def get_user_active_bookings(self, user_id: int):
```

```
    return [
```

```
        b for b in self.bookings.values()
        if b.user_id == user_id and b.status == BookingStatus.BOOKED
    ]
```

```
def get_user_bookings(self, user_id: int):
    return [b for b in self.bookings.values() if b.user_id == user_id]
```

```
def add_audit_event(self, event_type: str, user_id: int, slot_id: int, details: str = ""):
    event = AuditEvent(
        id=self._audit_id,
        event_type=event_type,
        user_id=user_id,
        slot_id=slot_id,
        timestamp=datetime.utcnow(),
        details=details,
    )
    self.audit_events[self._audit_id] = event
    self._audit_id += 1
    return event
```

```
def get_audit_events(self):
    return list(self.audit_events.values())
```

audit_service.py

```
class AuditService:
```

```
    def __init__(self, repo):
        self.repo = repo
```

```
    def log_booking_created(self, user_id: int, slot_id: int):
```

```
self.repo.add_audit_event(  
    event_type="BOOKING_CREATED",  
    user_id=user_id,  
    slot_id=slot_id,  
    details="Booking successfully created"  
)
```

```
def log_booking_cancelled(self, user_id: int, slot_id: int):
```

```
    self.repo.add_audit_event(  
        event_type="BOOKING_CANCELLED",  
        user_id=user_id,  
        slot_id=slot_id,  
        details="Booking successfully cancelled"  
    )
```

```
def log_booking_rejected(self, user_id: int, slot_id: int, reason: str):
```

```
    self.repo.add_audit_event(  
        event_type="BOOKING_REJECTED",  
        user_id=user_id,  
        slot_id=slot_id,  
        details=reason  
    )
```

```
def get_audit_log(self):
```

```
    return self.repo.get_audit_events()
```

```
schedule_service.py
```

```
from models import SlotStatus
```

```
class ScheduleError(Exception):
```

```
    pass
```

```
class ScheduleService:
```

```
    def __init__(self, repo):
```

```
        self.repo = repo
```

```
    def get_available_slots(self, specialist_id: int, date):
```

```
        result = []
```

```
        for slot in self.repo.slots.values():
```

```
            if (
```

```
                slot.specialist_id == specialist_id
```

```
                and slot.start_time.date() == date
```

```
                and slot.status == SlotStatus.AVAILABLE
```

```
            ):
```

```
                result.append(slot)
```

```
        return result
```

```
    def ensure_slot_available(self, slot_id: int):
```

```
        slot = self.repo.get_slot(slot_id)
```

```
        if not slot:
```

```
            raise ScheduleError("Slot does not exist")
```

```
        if slot.status != SlotStatus.AVAILABLE:
```

```
            raise ScheduleError("Slot is not available")
```

```
        return slot
```



```

def has_time_conflict(self, user_id: int, candidate_slot):
    user_bookings = self.repo.get_user_active_bookings(user_id)
    for booking in user_bookings:
        booked_slot = self.repo.get_slot(booking.slot_id)
        if booked_slot is None:
            continue
        overlaps = not (
            candidate_slot.end_time <= booked_slot.start_time
            or candidate_slot.start_time >= booked_slot.end_time
        )
        if overlaps:
            return True
    return False

```

```

def mark_slot_booked(self, slot):
    slot.status = SlotStatus.BOOKED
    self.repo.save_slot(slot)

```

```

def restore_slot_after_cancellation(self, slot):
    if slot.status != SlotStatus.BLOCKED:
        slot.status = SlotStatus.AVAILABLE
    self.repo.save_slot(slot)

```

booking_service.py

from datetime import datetime, timedelta

from models import BookingStatus

from schedule_service import ScheduleError

```
class BookingError(Exception):
```

```
    pass
```

```
class BookingService:
```

```
    def __init__(self, repo, schedule_service, audit_service):
```

```
        self.repo = repo
```

```
        self.schedule_service = schedule_service
```

```
        self.audit_service = audit_service
```

```
    def get_available_slots(self, specialist_id: int, date):
```

```
        return self.schedule_service.get_available_slots(specialist_id, date)
```

```
    def get_user_bookings(self, user_id: int):
```

```
        return self.repo.get_user_bookings(user_id)
```

```
    def book_slot(self, user_id: int, slot_id: int):
```

```
        try:
```

```
            slot = self.schedule_service.ensure_slot_available(slot_id)
```

```
        except ScheduleError as e:
```

```
            self.audit_service.log_booking_rejected(user_id, slot_id, str(e))
```

```
            raise BookingError(str(e))
```

```
    active_bookings = self.repo.get_user_active_bookings(user_id)
```

```
    if len(active_bookings) >= 3:
```

```
        self.audit_service.log_booking_rejected(user_id, slot_id, "User reached booking  
limit")
```

```
        raise BookingError("User reached booking limit")
```

```
if self.schedule_service.has_time_conflict(user_id, slot):  
    self.audit_service.log_booking_rejected(user_id, slot_id, "Time conflict detected")  
    raise BookingError("User already has overlapping booking")
```

```
self.schedule_service.mark_slot_booked(slot)  
  
booking = self.repo.create_booking(user_id, slot_id)  
  
self.audit_service.log_booking_created(user_id, slot_id)  
  
return booking
```

```
def cancel_booking(self, user_id: int, booking_id: int):  
    booking = self.repo.get_booking(booking_id)  
  
    if not booking:  
        raise BookingError("Booking does not exist")  
  
    if booking.user_id != user_id:  
        raise BookingError("User cannot cancel another user's booking")  
  
    if booking.status != BookingStatus.BOOKED:  
        raise BookingError("Booking is not active")  
  
    slot = self.repo.get_slot(booking.slot_id)  
  
    if not slot:  
        raise BookingError("Related slot does not exist")  
  
    if slot.start_time - datetime.utcnow() <= timedelta(hours=24):  
        raise BookingError("Cancellation too late")
```

```
booking.status = BookingStatus.CANCELLED

self.repo.save_booking(booking)

self.schedule_service.restore_slot_after_cancellation(slot)

self.audit_service.log_booking_cancelled(user_id, slot.id)

return booking
```

7. Klasy i odpowiedzialności

BookingService

- rezerwacja,
- anulowanie,
- egzekwowanie reguł biznesowych.

ScheduleService

- pobieranie dostępnych slotów,
- walidacja statusu slotu,
- wykrywanie konfliktów czasowych,
- zmiana statusu slotów.

AuditService

- rejestrowanie zdarzeń,
- logowanie odrzuconych operacji,
- udostępnianie historii zmian.

8. Wymagane właściwości gold code

Kod referencyjny powinien być:

- funkcjonalnie poprawny,
- modularny,
- łatwy do testowania,
- zgodny z requirements i architecture,

9. Co stanowi wynik referencyjny

Za poprawny gold code uznaje się implementację, która:

- pozwala pobrać dostępne sloty,

- egzekwuje limit 3 aktywnych rezerwacji,
- wykrywa nakładające się wizyty,
- odrzuca próbę rezerwacji zajętego slotu,
- obsługuje anulowanie zgodnie z regułą 24h,
- rejestruje zdarzenia audytowe,
- utrzymuje spójność statusów bookingów i slotów.

10. Do czego używać tego gold

Ten artifact może być użyty jako:

- gold code dla Coding & Refactoring Team,
- wejście lokalne dla Testing Team,
- część gold project dla DevOps Team,
- punkt odniesienia do oceny kodu generowanego przez LLM i studentów.

11. Uwaga techniczna

W wersji gold dopuszczalna jest implementacja uproszczona, np. InMemoryRepository, o ile zachowuje złożoność logiki biznesowej i modularny podział odpowiedzialności.